# Software Technical Specification: AI Video Dubbing Pipeline

**Document Version:** 1.0.0 **System Target:** WSL2 / Ubuntu (NVIDIA RTX 3090 24GB) **Repository:** jobryantai/aipipeline

## 1. EXECUTIVE ABSTRACT

This technical specification outlines the design, architecture, and deployment standards for an automated, end-to-end multi-speaker cross-lingual video dubbing system. Operating fully on local GPU hardware, the pipeline ingests Japanese video source files, performs dynamic speaker turn identification, extracts target speaker voice profiles, translates transcribed text into English, synthesizes speech with zero-shot voice cloning (XTTS-v2), and remuxes final synchronized audio into video outputs.

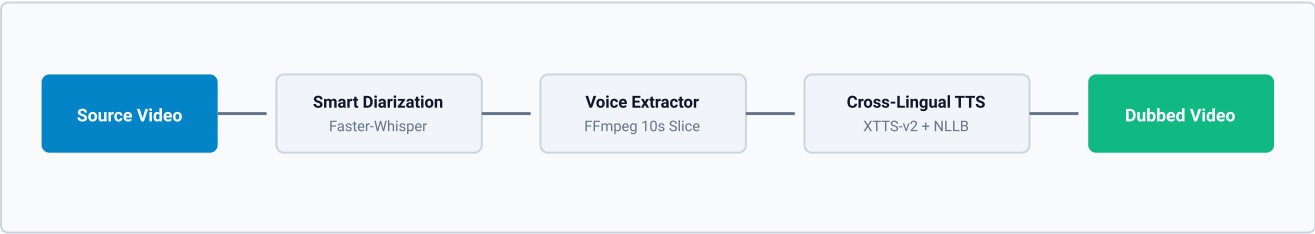
## 2. PROJECT GOALS & SCOPE

- **Local Zero-Shot Cloning:** Maintain original speaker voice acoustics in translated output without cloud dependencies.
- **Multi-Speaker Diarization:** Dynamically isolate dynamic turn-taking across distinct speaker identities.
- **Low Real-Time Factor (RTF):** Achieve high-throughput offline video processing (≤ 0.15 RTF) on single RTX 3090 hardware.

## 3. SYSTEM SETUP & DEPENDENCY SPECIFICATIONS

| Component | Specification / Version | Technical Purpose |
|---|---|---|
| Environment | WSL2 / Ubuntu 22.04 LTS | Linux POSIX compatibility under Windows Host |
| Compute Stack | CUDA 12.1 / PyTorch 2.5.1 | GPU Acceleration & Tensor Execution Engine |
| Core Libraries | NumPy 1.26.4 / Pandas 1.5.3 | Pinned matrix/array operations for Coqui TTS compatibility |
| ASR Engine | Faster-Whisper (large-v3) | High-accuracy Japanese speech transcription |
| Translation | NLLB-200 Distilled 600M | Neural Machine Translation ( `jpn_Jpan` → `eng_Latn` ) |
| TTS & Cloning | Coqui XTTS-v2 | Multi-speaker zero-shot cross-lingual speech synthesis |
| Media Processing | FFmpeg 4.4+ / SoundFile | Audio slice extraction, channel demuxing, and final remuxing |

## 4. END-TO-END SYSTEM ARCHITECTURE FLOWCHART




## 5. INPUT / OUTPUT DATA CONTRACTS

| Interface Stage | Input Data / Schema | Output Artifact |
|---|---|---|
| Stage 1: ASR & Diarization | `my_video.mp4` (Container: H.264 / AAC) | `segments_diarized.json` (Array: start, end, text, speaker) |
| Stage 1.5: Reference Extraction | `segments_diarized.json` + `my_video.mp4` | `speaker_refs/{SPEAKER_ID}.wav` (PCM 16-bit, 22.05kHz mono) |
| Stage 2: Translation & TTS | `segments_diarized.json` + Reference WAVs | `audio_en_synced.wav` (PCM 32-bit float, 24kHz master) |
| Stage 3: Video Assembly | `my_video.mp4` + `audio_en_synced.wav` | `final_dubbed_video.mp4` (AAC audio overlayed with video) |

## 6. TECHNICAL LIMITATIONS & CONSTRAINT BOUNDARIES

**System Constraints:**

- **Dependency Pinning:** Upgrading `numpy > 1.26.4` causes C-API ABI mismatch crashes with Coqui TTS engine.
- **Acoustic Isolation:** Background music or noisy audio within source speaker reference segments degrades voice clone quality. Clean 6–10 second voice samples are required.
- **Temporal Alignment:** Direct translation can lead to English target phrases exceeding original Japanese video window duration.